SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**
Total Marks:30

Annexure A

Dry Run Evaluation

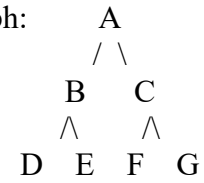
Code of Conduct:

I **Kamali S** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Kamali S(192524423)**

1.Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1	Before: [A] **After: [B, C]**	A
2	Before: [B, C] **After: [C, D, E]**	B
3	Before: [C, D, E] **After: [D, E, F, G]**	C
4	Before: [D, E, F, G] **After: [E, F, G]**	D
5	Before: [E, F, G] **After: [F, G]**	E
6	Before: [F, G] **After: [G]**	F
7	Before: [G] **After: [ ] (empty)**	G

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer 1 – BFS traversal order

BFS traversal order: A → B → C → D → E → F → G

Breadth-First Search explores a graph level by level. It keeps a **FIFO (First-In-First-Out) queue** of nodes that have been discovered but not yet visited. The node at the front of the queue is always removed next, and its unvisited neighbours are added at the back. Because of this, every node at distance k from the start is visited before any node at distance k + 1. In the given tree, Level 0 contains A, Level 1 contains B and C, and Level 2 contains D, E, F and G. Visiting level by level therefore gives A, then B C, then D E F G, which is exactly the order above.

Step-by-step dry run

Initialisation: the queue contains only the start node, [A]. Nothing has been visited yet.

Iteration 1: remove A from the front and visit it. Its children are B and C (left to right), both unvisited, so enqueue them. Queue: before [A], after [B, C].

Iteration 2: remove B and visit it. Its children D and E are enqueued behind C. Queue: before [B, C], after [C, D, E]. Note that C stays ahead of D and E, which is what keeps the search level by level.

Iteration 3: remove C and visit it. Its children F and G are enqueued. Queue: before [C, D, E], after [D, E, F, G]. All of Level 1 is now finished, and the queue holds exactly the whole of Level 2.

Iteration 4: remove D and visit it. D is a leaf (no children), so nothing is added. Queue: [E, F, G].

Iteration 5: remove E and visit it. E is a leaf, so nothing is added. Queue: [F, G].

Iteration 6: remove F and visit it. F is a leaf. Queue: [G].

Iteration 7: remove G and visit it. G is a leaf. The queue is now empty, so the algorithm terminates after visiting all 7 nodes.

Answer 2 – Queue contents after each iteration

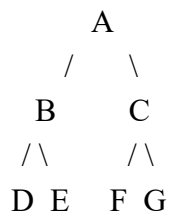
The queue contents after every iteration (front of the queue on the left) are listed below and also filled in the table above.

- After iteration 1: **[B, C]**
- After iteration 2: **[C, D, E]**
- After iteration 3: **[D, E, F, G]**
- After iteration 4: **[E, F, G]**
- After iteration 5: **[F, G]**
- After iteration 6: **[G]**
- After iteration 7: **[ ] (empty)**

Observations

- The queue is largest after iteration 3 (4 nodes), because it then holds the entire widest level of the tree. This is why BFS needs memory proportional to the width of the graph, O(b^d).
- Nodes leave the queue in the same order in which they entered it, so the visiting order is the same as the order of insertion. That is the defining property of a FIFO queue.
- Each node is enqueued once and dequeued once and each edge is examined once, so the time complexity is O(V + E). Here V = 7 and E = 6.
- BFS is complete (it reaches every node of a finite graph) and optimal for unweighted graphs, because it reaches every node by a path with the fewest edges.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1	[B, C]	A
2	[D, E, C]	B
3	[E, C]	D
4	[C]	E
5	[F, G]	C
6	[G]	F
7	[ ]	G

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

Answer – DFS dry run

DFS traversal order: A → B → D → E → C → F → G

Depth-First Search goes as deep as possible along one branch before backtracking. It uses a **LIFO (Last-In-First-Out) stack**. When a node is expanded, its children are pushed so that the **left child ends up on top** (the right child is pushed first). The stack in the table is written with its **top on the left** and shows the contents **after** each iteration.

Step-by-step dry run

Initialisation: push the start node. Stack: [A].

Iteration 1: pop A and visit it. Push its children C first and then B, so B is on top. Stack: [B, C].

Iteration 2: pop B and visit it. Push E, then D. Stack: [D, E, C]. The search has now gone down the left branch to depth 2.

Iteration 3: pop D and visit it. D is a leaf, so nothing is pushed. Stack: [E, C].

Iteration 4: pop E and visit it. E is also a leaf. Stack: [C]. The whole left subtree of A is now finished, so the search has effectively backtracked to A and continues with its other child, C.

Iteration 5: pop C and visit it. Push G, then F. Stack: [F, G].

Iteration 6: pop F and visit it. F is a leaf. Stack: [G].

Iteration 7: pop G and visit it. G is a leaf and the stack becomes empty, so the search ends having visited all 7 nodes.

Observations and comparison with BFS

- The maximum stack size is 3 (after iteration 2). DFS needs memory only proportional to the depth of the tree (O(b·m)), whereas BFS needed to hold a whole level (4 nodes here). This makes DFS much cheaper in memory on large graphs.
- Time complexity is O(V + E), the same as BFS, since every node and edge is processed once.
- DFS is **not optimal**: it returns the first path it finds, which need not be the shortest. It can also fail to terminate on infinite or cyclic graphs unless a visited set or depth limit is used.
- The order depends on the order in which children are pushed. If the right child were pushed last (so it sits on top), the order would become A → C → G → F → B → E → D.

Feature	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO)
Order on this graph	A B C D E F G	A B D E C F G
Maximum frontier size here	4 nodes	3 nodes
Space complexity	O(b^d)	O(b·m)
Finds shortest path (unweighted)	Yes	Not guaranteed

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	**S0** 1 3 6 5 0 2 4 7 8	S0	S1, S2, S3, S4
2	**S1** 1 0 6 5 3 2 4 7 8	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	**S2** 1 3 6 5 7 2 4 0 8	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	**S3** 1 3 6 0 5 2 4 7 8	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	**S4** 1 3 6 5 2 0 4 7 8	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Problem set-up used for the trace

Each state is a 3 × 3 arrangement of the tiles 1–8 and the blank (0). The only operator is to slide the blank **Up, Down, Left or Right**, which has the effect of moving a neighbouring tile into the gap. Every move has cost 1. The blank in a corner has 2 legal moves, on an edge 3 moves and in the centre 4 moves. Children are generated in the fixed order **Up, Down, Left, Right**. BFS uses a FIFO queue, and a state that has already been generated is not added again (a repeated-state check), otherwise the search would keep going back and forth. The goal test is applied when a state is removed from the queue.

To keep the table short, states are given labels S0, S1, S2, … in the order in which they are generated. The key is shown below.

State	Configuration	Generated from / by move
S0	1 3 6 / 5 0 2 / 4 7 8	Start state
S1	1 0 6 / 5 3 2 / 4 7 8	S0: blank Up (tile 3 moves down)
S2	1 3 6 / 5 7 2 / 4 0 8	S0: blank Down (tile 7 moves up)
S3	1 3 6 / 0 5 2 / 4 7 8	S0: blank Left (tile 5 moves right)
S4	1 3 6 / 5 2 0 / 4 7 8	S0: blank Right (tile 2 moves left)
S5	0 1 6 / 5 3 2 / 4 7 8	S1: blank Left
S6	1 6 0 / 5 3 2 / 4 7 8	S1: blank Right
S7	1 3 6 / 5 7 2 / 0 4 8	S2: blank Left

State	Configuration	Generated from / by move
S8	1 3 6 / 5 7 2 / 4 8 0	S2: blank Right
S9	0 3 6 / 1 5 2 / 4 7 8	S3: blank Up
S10	1 3 6 / 4 5 2 / 0 7 8	S3: blank Down
S11	1 3 0 / 5 2 6 / 4 7 8	S4: blank Up
S12	1 3 6 / 5 2 8 / 4 7 0	S4: blank Down

The three rows of each configuration are separated by "/". Moving the blank back to where it came from would recreate the parent, so that child is discarded as a repeat. This is why S1, S2, S3 and S4 each add only 2 new states, although the blank has 3 legal moves in each of them.

Answer 1 – Which node is expanded in each iteration?

- **Iteration 1:** S0, the initial state, is expanded. Its four children S1, S2, S3 and S4 are generated and added to the queue.
- **Iteration 2:** S1 is expanded (first in the queue). It produces S5 and S6 and the move back to S0 is rejected as a repeat.
- **Iteration 3:** S2 is expanded and produces S7 and S8.
- **Iteration 4:** S3 is expanded and produces S9 and S10.
- **Iteration 5:** S4 is expanded and produces S11 and S12.

Because the queue is FIFO, the states are expanded strictly in the order they were generated, so the 5 rows of the table are S0, S1, S2, S3 and S4. Iterations 2 to 5 expand all four depth-1 states, after which the search goes on with S5, S6, … (depth 2), and so on.

To reach the goal, BFS has to continue past iteration 5. The nodes on the final solution path are removed from the queue at iterations **1, 5, 12, 20, 34, 63, 119 and 187**, and the **goal state itself is removed at iteration 311**, where the goal test succeeds. So 310 states are fully expanded before the goal is reached.

Answer 2 – How many states are generated in total?

After the 5 iterations in the table: 1 (initial state) + 4 + 2 + 2 + 2 + 2 = **13 states** (S0 to S12).

Until the goal is reached: counting the initial state, **500 states are generated** (499 produced by expansion plus the start state) by the time the goal is removed from the queue at iteration 311. Of these, 311 have been taken from the queue (including the goal) and 189 are still waiting in it. The generated states are spread over the depths as follows:

Depth	States generated	Remark
0	1	Initial state
1	4	Children of S0
2	8	
3	8	
4	16	
5	32	
6	60	
7	72	
8	136	Contains the goal state
9	163	Generated while finishing depth 8
Total	500	

If the goal test were applied the moment a state is generated instead of when it is removed, the goal would be recognised as the 311th generated state (during the 187th expansion), so fewer states would be needed. The numbers above use the standard textbook rule of testing on removal.

Answer 3 – Complete solution path

Solution: Right → Up → Left → Down → Left → Down → Right → Right (moves of the blank). The path has **8 moves**, so the solution cost is **8**.

Start	1. Right	2. Up	3. Left	4. Down	5. Left	6. Down	7. Right	8. Right
1 3 6 5 0 2 4 7 8	1 3 6 5 2 0 4 7 8	1 3 0 5 2 6 4 7 8	1 0 3 5 2 6 4 7 8	1 2 3 5 0 6 4 7 8	1 2 3 0 5 6 4 7 8	1 2 3 4 5 6 0 7 8	1 2 3 4 5 6 7 0 8	1 2 3 4 5 6 7 8 0

In terms of tiles, the sequence is: slide tile 2 left, tile 6 down, tile 3 right, tile 2 up, tile 5 right, tile 4 up, tile 7 left, and finally tile 8 left. Each arrow below is one move of the blank:

(1 3 6 / 5 0 2 / 4 7 8) → (1 3 6 / 5 2 0 / 4 7 8) → (1 3 0 / 5 2 6 / 4 7 8) → (1 0 3 / 5 2 6 / 4 7 8) → (1 2 3 / 5 0 6 / 4 7 8) → (1 2 3 / 0 5 6 / 4 7 8) → (1 2 3 / 4 5 6 / 0 7 8) → (1 2 3 / 4 5 6 / 7 0 8) → (1 2 3 / 4 5 6 / 7 8 0), where each group is written row by row.

In the BFS tree this path is S0 → S4 → S11 → … , the first branch leaving S0 through the move Right and then Up. The goal lies at depth 8.

Answer 4 – Why BFS always finds the shortest solution

BFS finds the shortest solution in an unweighted 8-puzzle for the following reasons:

- **Path cost equals depth.** Every move slides one tile and costs exactly 1, so the cost of a path is simply the number of moves, which is the depth of the node in the search tree. The shortest solution is therefore the shallowest goal.
- **The FIFO queue forces level-by-level expansion.** The queue first holds depth 0, then all depth-1 states, then all depth-2 states, and so on. Children are always added behind nodes of their parent's level, so nodes are removed in non-decreasing order of depth. All nodes at depth d are expanded before any node at depth d + 1.
- **The first time the goal is reached is at the smallest depth.** Suppose a shorter solution of depth k existed while BFS returned a solution of depth m > k. The goal at depth k would have been generated while expanding depth k − 1, and, being in the queue ahead of every depth m node, it would have been removed first. This contradicts the claim, so no shorter solution exists.
- **The repeated-state check does not hurt.** A state is generated for the first time through a shortest route to it, because shallower levels are processed first. Discarding later copies discards only longer routes.
- **Completeness.** The state space of the 8-puzzle is finite and the branching factor is at most 4, so BFS cannot miss a reachable goal. This puzzle is solvable, since the initial state has an even number of inversions (6), the same parity as the goal.
- **Independent check.** The sum of the Manhattan distances of the tiles from their goal positions in the initial state is 0 + 1 + 1 + 1 + 2 + 1 + 1 + 1 = 8. No solution can use fewer than 8 moves, and BFS found one with exactly 8, which confirms that it is optimal.

Cost of this guarantee: the time and memory of BFS grow as O(b^d). In this problem it had to generate 500 states to find a depth-8 solution, and for the hardest 8-puzzle instances (depth 31) the number of states is very large. Informed search such as A* with the Manhattan-distance heuristic keeps the same optimality while expanding far fewer nodes. When all step costs are equal, Uniform-Cost Search behaves exactly like BFS.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.